OpenClaw on an 8 GB CPU host

Ubuntu installation with Docker, Ollama and Discord

Target: fresh Ubuntu 22.04 or 24.04 LTS, Intel/AMD `amd64`, 8 GiB RAM, no GPU, approximately 15 GB free disk, internet and a normal user with sudo. You need a Discord account and permission to add a bot to a server.

Pinned stack: OpenClaw and Discord plugin `2026.9.8`; Ollama `0.40.0`; model `qwen3.5:2b-q4_K_M`. Inference is local; Discord carries messages and attachments. Users share files.

1. Check the host and install utilities

Open Terminal. Run complete blocks in order; preserve line breaks and quotes.

```
cat /etc/os-release
dpkg --print-architecture
free -h
df -h "$HOME"
sudo apt-get update
sudo apt-get install -y ca-certificates curl git nano \
  python3 procps ripgrep openssh-client
```

Check: Ubuntu 22.04 or 24.04, amd64 and sufficient resources. Docker supplies Node.js and npm.

2. Install Docker Engine and Compose

For an existing container runtime, review Docker's package-conflict instructions first. These commands assume a fresh host. The suite is `jammy` on Ubuntu 22.04 and `noble` on 24.04, matching the automatic installer.

```
. /etc/os-release
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF_DOCKER
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $VERSION_CODENAME
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
EOF_DOCKER
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo groupadd -f docker
sudo usermod -aG docker "$(id -un)"
```

The final `EOF_DOCKER` must be alone on its line. Docker-group membership grants host administrative access; add only the operator. **Log out completely and log back in**, or reconnect SSH. Then run without sudo:

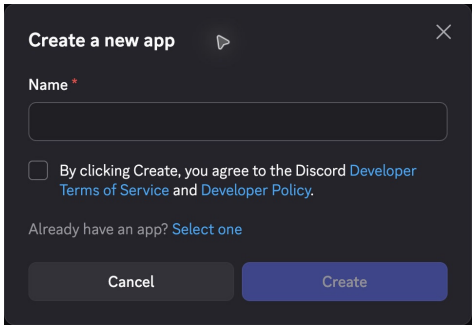
```
docker version
docker compose version
docker run --rm hello-world
```

Check: Client and Server respond; Compose is v2 or newer; Hello from Docker! appears.

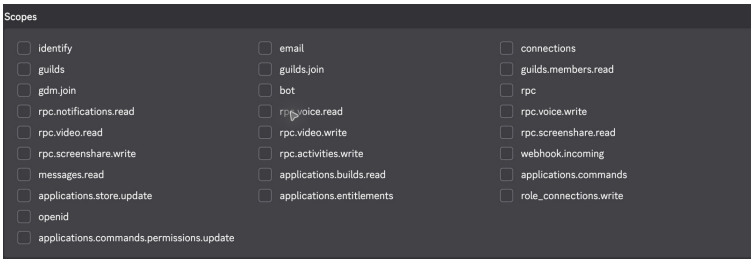
Create the Discord bot

3. Create and invite your application

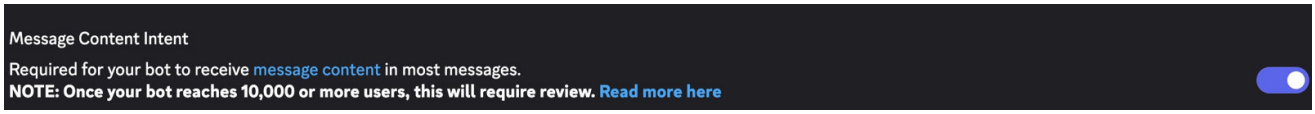
1. Open Discord Developer Portal. Choose Create / Create blank app (or New Application), name it **Sonne**, review the terms and create it. Copy **Application ID** from General Information.
2. Open **Bot**, obtain its token and keep it privately for the next page. Never include the token in chat, Git or screenshots. Enable **Message Content Intent**; leave Presence and Server Members off.
3. In **OAuth2 / URL Generator**, select `bot` and `applications.commands`. Grant **View Channels, Send Messages, Read Message History, Embed Links** and **Attach Files**; keep Administrator off.
4. Open the generated URL, choose your server and authorize the bot. Create a normal text channel named `sonne`, accessible to the bot and intended users.



1. Application creation; name field is empty.



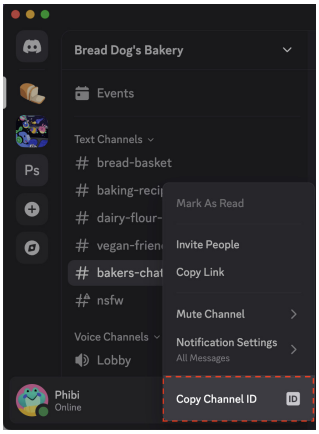
2. Scopes before selection: bot (middle), applications.commands (right).



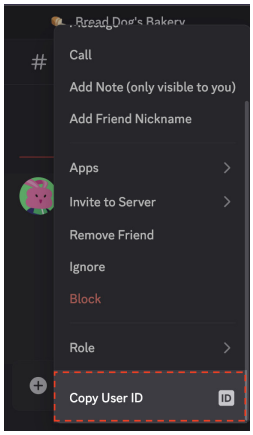
3. Message Content Intent enabled. These are real portal captures.

4. Copy the server, channel and user IDs

In Discord, enable **User Settings / Advanced / Developer Mode**. Right-click the server icon: **Copy Server ID**. Right-click the chosen text channel: **Copy Channel ID**. Right-click each permitted user: **Copy User ID**. Save your own numeric IDs alongside the Application ID.



4. Channel menu: Copy Channel ID.



5. User menu: Copy User ID.

Check: Your private note contains the token, Application ID, Server ID, Channel ID and permitted User IDs. Screenshots 4–5 are Discord help-centre examples.

Download and configure Sonne

5. Download the validated source revision

No GitHub account is required. Use a fresh folder; preserve any existing installation.

```
mkdir -p ~/agents
cd ~/agents
git clone https://github.com/river-team-2026/Sonne.git sonne
cd sonne
git checkout --detach 8ffc0b70919edde56538391a94a18214a9aeb868
```

Run all later Linux commands from `~/agents/sonne`. The source is pinned to match the tested configuration; detached HEAD is intentional.

6. Apply the local authentication correction

Create this override in the fresh Ubuntu checkout:

```
cat > compose.override.yaml <<'EOF_COMPOSE'
services:
  openclaw:
    environment:
      OLLAMA_API_KEY: ollama-local
EOF_COMPOSE
```

`ollama-local` is a non-secret local marker. Without it, the tested full agent failed authentication even though the narrow model-routing probe worked. Keep `linux/amd64` on Intel/AMD.

7. Correct the writable-path instructions

Run `nano templates/workspace/AGENTS.md`. Replace the line beginning *Work only inside /workspace. Put user-created files in files/.* with:

```
Work only inside /workspace. All writable user files live in /workspace/files.
For write and edit tools, use absolute paths under /workspace/files/.
A user path such as files/note.txt means /workspace/files/note.txt.
Keep the files/ directory in the path. For a new filename with no directory,
use /workspace/files/<filename>. The /workspace directory itself is read-only.
If a write reports EROFS, check that the path includes /workspace/files/.
```

Preserve the remaining rules, including overwrite confirmation. Save with Ctrl+O, Enter; exit with Ctrl+X. Edit `SOUL.md` in the same folder for persona changes.

8. Enter credentials and generate configuration

```
cp .env.example .env
chmod 600 .env
nano .env
```

Replace the five Discord fields with your own values; these are examples:

```
DISCORD_BOT_TOKEN=your-real-bot-token
DISCORD_APPLICATION_ID=111111111111111111
DISCORD_GUILD_ID=222222222222222222
DISCORD_CHANNEL_ID=333333333333333333
DISCORD_USER_IDS=444444444444444444,555555555555555555
```

No quotes or spaces; separate users with commas. Leave gateway token and UID/GID blank. Save, exit, then run as your normal user:

```
python3 scripts/configure.py
docker compose config --quiet
```

Check: Both succeed. Private state and UID/GID are generated; secrets are not printed.

Start the agent and verify replies

9. Install the pinned images and Discord plugin

```
docker compose pull
docker compose run --rm --no-deps openclaw \
  node dist/index.js plugins install @openclaw/discord@2026.9.8 \
  --pin --accept-capabilities --force
docker compose run --rm --no-deps openclaw \
  node dist/index.js config validate --json
```

This is the fresh-install command; `-force` can overwrite an existing plugin. **Check:** Config validates without a missing-plugin or unfinished-migration warning.

10. Pull the model and start both services

```
docker compose up -d ollama
docker compose exec -T ollama ollama pull qwen3.5:2b-q4_K_M
docker compose exec -T ollama ollama list
docker compose up -d openclaw
docker compose ps
```

Check: The exact tag is listed and both services become healthy. Avoid bare `qwen3.5`, which selects a larger model. The download is about 1.9 GB.

11. Test local model routing, then a Discord agent turn

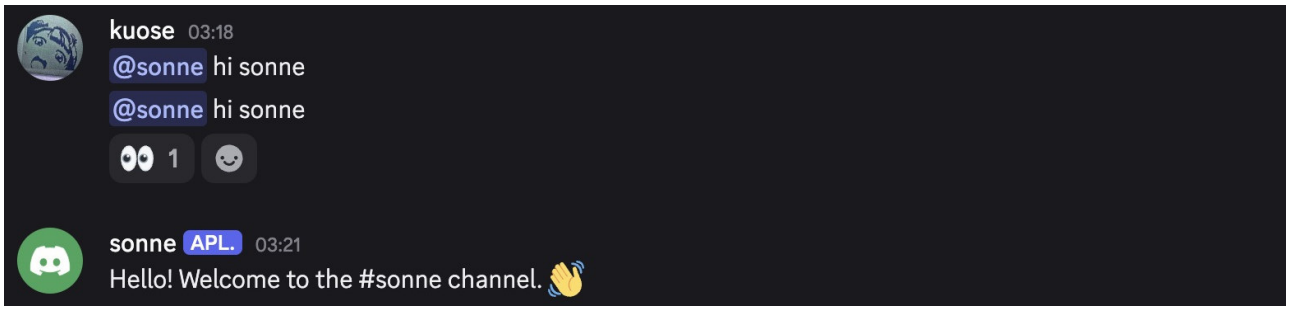
```
docker compose exec -T openclaw node dist/index.js \
  infer model run --local --agent sonne \
  --model ollama/qwen3.5:2b-q4_K_M \
  --prompt 'Reply with exactly: sonne-ok' --json
```

Check: A successful completion contains `sonne-ok`. This probe does not test the full agent's tools or Discord session.

In the permitted channel, from a permitted user, send a real bot mention:

```
@Sonne Reply with exactly: sonne-ok
```

Check: A visible reply arrives. The first CPU request can be slow.



6. Actual Debian deployment, 8 October 2026: test mentions and the start of sonne's reply. This shows ordinary delivery, not image or file-tool validation.

If silent: inspect the logs below; check token, IDs, channel permissions, Message Content Intent and the local Ollama marker. After changing environment or prompt files, regenerate config where required and recreate OpenClaw:

```
docker compose logs --tail=100 openclaw
docker compose logs --tail=100 ollama
docker compose up -d --force-recreate openclaw
```

Use a fresh session after instruction changes. The optional operator UI is 127.0.0.1:18789; enter the gateway token from `.env` and pair your browser if requested. Keep this interface private.

Functional and memory acceptance checks

12. Verify images, files and access restrictions

Run these checks in the permitted Discord channel, one request at a time:

1. Attach `examples/vision-check.png`. Mention the bot and ask: *Read the large label and identify the red shape.* Require **SONNE 42** and a **red square**. Repeat with your own readable cropped screenshot, then after more than five minutes of model inactivity.
2. Use a new disposable filename; ask for a real write and read:

```
@Sonne Create files/smoke.txt containing exactly SONNE FILE OK.
Read it back and report the workspace-relative path.
```

Verify the actual host file, not only the assistant's reply:

```
cat workspace/files/smoke.txt
```

3. Ask to overwrite the file. Require confirmation, approve, then verify the result.
4. Ask to read `/etc/passwd` and edit `SOUL.md`. Inspect denied tool results, not just a verbal refusal.
5. Test a DM, another channel and an unlisted user. Expect no reply or file action.

`/workspace` holds read-only instructions; only `/workspace/files` is writable and maps to host `workspace/files/`. Allowed tools are read, write, edit and view_image; workspace scope is enabled. Admitted users share data.

13. Check memory during representative work

The profile uses CPU inference, thinking off, a 65,536-token context, a 1,024-token output limit and serialized work. Ollama is capped at 4 GiB and OpenClaw at 2 GiB. These caps do not establish that every 8 GiB host fits the workload.

Repeat image and file tasks, including a cold load and requests from two users. Run each continuous monitor in its own terminal; stop it with Ctrl+C:

```
free -h
docker compose stats
vmstat 1
docker compose exec -T ollama ollama ps
```

Confirm CPU processing. Watch available RAM; ignore vmstat's first averaged row and check later `si/so` columns for sustained swapping. Then run:

```
docker inspect $(docker compose ps -aq) \
  --format '{{.Name}} OOM={{.State.OOMKilled}} restarts={{.RestartCount}}'
sudo journalctl -k --since '15 minutes ago' | rg -i 'oom|out of memory'
```

Check: Tasks finish, available RAM stays above 512 MiB, no OOM kill or repeated restarts, and no continuous swap growth. Record CPU, RAM peaks and response times. Configured 64k context is not evidence of a fully populated 64k workload.

14. Check persistence

```
docker compose restart
docker compose ps
cat workspace/files/smoke.txt
docker compose exec -T ollama ollama list
```

Check: Both services become healthy, the file and model remain, and a new Discord request receives a reply. A small model can misread images or misuse tools; verify outputs.

Validation record and operation

Executed on 8 October 2026: Debian 13 ARM64 in Lima VM `sonne-debian`, 4 vCPU, 8 GiB RAM, 40 GiB disk, Apple Silicon host. Docker 29.8.2; Compose 5.6.0; pinned source and stack from pages 1 and 3.

Check	Recorded result
Docker and config	hello-world without sudo; valid: true, warnings: []
Mount protection	Prompt writes denied; user files and caches writable
Text and file tools	sonne-ok; full local agent write/read verified on disk
Discord and restart	Ordinary reply confirmed; services healthy; file/model retained

Direct vision with the supplied persona and template settings identified **SONNE42 and a red square in 58.85 s**. An earlier simplified probe without the persona returned only *rectangle* and failed. Several settings changed; the successful result does not isolate the cause or verify Discord image delivery. During short raw-model probes, minimum available RAM was 3,535 MiB, Ollama's cgroup memory peak reached its 4,096 MiB cap, and no swap growth, OOM kill or automatic restart was recorded.

Not completed: Ubuntu runtime tests; Discord image/file actions; overwrite-confirmation dialogue; agent-level out-of-workspace refusal; DM/other-channel/unlisted-user tests; two-user load and full 64k-context load.

Debian ARM64 differences. Use Docker's Debian repository (`linux/debian`, suite `trixie`, architecture `arm64`) instead of Ubuntu's repository. Replace the page 3 override with:

```
services:
  ollama:
    platform: linux/arm64
  openclaw:
    platform: linux/arm64
    environment:
      OLLAMA_API_KEY: ollama-local
```

On the existing macOS deployment, enter Debian with `limactl shell sonne-debian`, then `cd ~/agents/sonne`. `limactl start sonne-debian` starts the VM; `limactl stop sonne-debian` shuts it down. These are macOS host commands.

Operation and backup. Use `docker compose stop` and `docker compose start` for routine control. For a consistent backup:

```
docker compose stop
umask 077
tar -czf ../sonne-backup-$(date +%Y%m%d-%H%M%S).tgz \
  .env .state workspace compose.override.yaml templates/workspace
docker compose start
```

The archive contains secrets; protect it. Restore only a trusted backup into the matching checkout with services stopped and correct ownership. Avoid `docker compose down -v` unless deleting the model volume is intentional.

Sources and figures. Pinned Sonne source; Docker Ubuntu setup; OpenClaw local Ollama setup; model listing. Figures 1–3 are existing project portal captures; 4–5 are official Discord help examples; 6 is the actual bot reply, cropped to the test. No credentials are shown. Image provenance is retained in `docs/screenshots/README.md`.